

COMPUTER SCIENCE MENTORS 61A

April 28, 2026 – May 1, 2026

Example Timeline

- Teaching tips:

Each problem's "Suggested Time" is not assigned with the 1 hour time for sections in mind. It is more of a note of how long each individual problem may take. That is, if you added all the "Suggested Times" in the coding section, it could very well be greater than 1 hour (especially in more complicated worksheets in the future).

However, the time in brackets will add up to 50 minutes, and is intended to be a suggested outline for your section.

Also, the worksheet feedback form is linked at the bottom of this worksheet!

- Intros, Ice Breakers, & Logistics [2 min]

Start off with an icebreaker of your choice!

- Mini-lecture: SQL Tables and Aggregation
- Q1: Pets [10 min]
- Q1: Mentors [20 min]
- Q2: Outfits [20 min]

1 SQL

1.

person	name	type	age
Alice	Buddy	Dog	3
Bob	Oreo	Cat	6
Cindy	Biscuit	Dog	4
David	Tom	Cat	2
Emily	Shelly	Tortoise	40

- (a) From the above table (pets), Write a query that gets the top two oldest pet's name and age.

```
Shelly|40  
Oreo|6
```

```
SELECT name, age FROM pets ORDER BY age DESC LIMIT 2
```

- (b) Write a query that gets the average age of each type of pet, and displays the rows in alphabetical order of the type of pet.

Dog | 3.5

Cat | 4

Tortoise | 40

```
SELECT name, AVG(age) FROM pets GROUP BY type ORDER BY type
```

Suggested Time: 10 minutes This question is easier, so if students are confident in SQL basics, you can skip to the next two questions!

CSM 61A Content Team wants to put together an end of the year party for all its mentors (and some special guests, too!) themed around everyone's favorite things! Examine the table, `mentors`, depicted below, and answer the following questions.

name	food	color	editor	language	animal
Alyssa	Pork Bulgogi	Navy Blue	Vim	Java	Sea Otter
Vibha	Pasta	Teal	VSCode	Java	Naked Mole Rat
Adit	Protein Bar	Black	Vim	Python	Gorilla
Esther	Goldfish	Pastel Pink	VSCode	Python	Bunny
Aiko	Fries	Sky Blue	VSCode	Java	Cat
Aurelia	Dumpling	Pastel Yellow	VSCode	Python	Panda
Kaelyn	Popcorn	Blue	VSCode	Java	Panda

- (a) Write a query that has the same data, but alphabetizes the rows by name. (Hint: Use **order by**.)

```
Adit|Protein Bar|Black|Vim|Python|Gorilla
Aiko|Fries|Sky Blue|VSCode|Java|Cat
Alyssa|Pork Bulgogi|Navy Blue|Vim|Java|Sea Otter
Aurelia|Dumpling|Pastel Yellow|VSCode|Python|Panda
Esther|Goldfish|Pastel Pink|VSCode|Python|Bunny
Kaelyn|Popcorn|Blue|VSCode|Java|Panda
Vibha|Pasta|Teal|VSCode|Java|Naked Mole Rat
```

```
SELECT *
FROM mentors
ORDER BY name;
```

- (b) Write a query that lists all the mentors along with their favorite food if their favorite color is green.

```
Matthew|Pie
Ivan|Ramen
```

```
select name, food
from mentors
where color = 'Green';

-- With aliasing
select m.name, m.food
from mentors as m
where m.color = 'Green';
```

- (c) Write a query that lists the food and the color of every person whose favorite language is *not* Python.

```
Pork Bulgogi|Navy Blue
Pasta|Teal
Fries|Sky Blue
Popcorn|Blue
```

```
SELECT food, color
FROM mentors
WHERE language != 'Python';
```

```
-- With aliasing (treating the table as a Python object) --
SELECT m.food, m.color
FROM mentors as m
WHERE m.language <> 'Python';
```

This is a simpler SQL problem that serves as a status check for if your students are caught up with SQL. Might be good to recap SQL keywords such as *select*, *from*, *where*, *order by*, and *limit* here.

- (d) Write a query that lists all the pairs of mentors who like the same language. (How can we make sure to remove duplicates?)

```
Catherine|Jamie
Ethan|Kenny
```

```
select m1.name, m2.name
from mentors as m1, mentors as m2
where m1.language = m2.language and m1.name < m2.name;
```

Some key points to explain:

- Why do we need to select from the same table twice? (To get pairs of mentors)
- Why do we need to alias here? Can we do it without aliasing? (No, we need aliasing because it is the same table)
- Why do we need `m1.name ; m2.name` ? Can we use `!=` instead or leave that out completely?

Suggested Time: 20 minutes

In this question, you have access to two tables.

Grades, which contains three columns: **day**, **class**, and **score**. Each row represents the score you got on a midterm for some **class** that you took on some **day**.

Outfits, which contains two columns: **day** and **color**. Each row represents the **color** of the shirt you wore on some **day**. Assume you have a row for each possible day.

grades

Day	Class	Score
10/31	Music 70	88
9/20	Math 1A	72

outfits

Day	Color
11/5	Blue
9/13	Red
10/31	Orange

- (a) You want to find out how you did on the midterms to which you wore your blue shirt to. So write a query which is going to yield a table where there is a row for each day you wore a blue shirt to a midterm (or multiple midterms) and then sum of midterm scores for that day.

```
SELECT grades.day, SUM(score)
FROM grades, outfits
WHERE outfits.color = "blue" and outfits.day = grades.day
GROUP BY grades.day;
```

- (b) You want to find out which classes you need to prepare for the most by determining how many points you have so far. However, you only want to do so for classes where you did relatively poorly.

Write a query that will output the sum of your midterm scores for each class along with the corresponding class, but only for classes in which you scored less than 80 points on at least one midterm. List the output from highest to lowest total score.

```
SELECT SUM(score), class
FROM grades GROUP BY class
HAVING MIN(score) < 80 ORDER BY SUM(score) DESC;
```

Teaching Tips

- Remind students that the **'HAVING'** clause is used to filter out groups (you cannot use **WHERE** on the groups).
- Remind students that aggregation works on each group individually; this means we want to **GROUP BY** the things we want to **SUM** over.

- (c) Instead of actually studying for your finals, you decide it would be the best use of your time to determine what your "lucky shirt" is. Suppose you're pretty happy with your exam scores this semester, so you define your lucky shirt as the shirt you wore to the most exams.

Write a query that will output the color of your lucky shirt and how many times you wore it.

```
SELECT color, COUNT(g.day) AS cnt
FROM outfits AS o, grades AS g
WHERE o.day = g.day
GROUP BY color
ORDER BY cnt DESC
LIMIT 1;
```

Teaching Tips

- Ensure students know all potential parts of a SQL query and how they work:
`SELECT [ ] FROM [ ] WHERE [ ] GROUP BY [ ] HAVING [ ] ORDER BY [ ] LIMIT [ ];`
- Remind students about the **SUM**, **MIN**, and **COUNT** functions
- Students may struggle with the variable to use in **ORDER BY**
 - Teach students they can alias created columns in **SELECT**
 - In SQL **SELECT** occurs before **ORDER BY**, so `cnt` can be used

Suggested Time: 20 minutes